

# A Five-Stage MIPS Pipeline in Anvil: Design, Specification, and a Type-System Evaluation

Achyuta Ganesh

**Research Question:** *How much of the safety specification that a hand-written SystemVerilog (SV) pipeline needs in hand-written SVA does Anvil's timing-safe type system give for free – and precisely where does that guarantee run out? The project answers this by building the same five-stage MIPS pipeline twice (SV, then Anvil), specifying the SV design in SVA, and testing the two implementations against each other with matched stimulus.*

## Part A – The SystemVerilog Design

The design is a classic five-stage pipeline (IF/ID/EX/MEM/WB) over a five-instruction subset of MIPS32. This processor supports load word (lw), store word (sw), jump on zero (jz), addition (add), and subtraction (sub) instructions. This was chosen as it was the smallest ISA that still forces every hazard class that is supported.

The design supports two modes of Forwarding - EX/MEM to EX and MEM/WB to EX, with an explicit priority (the youngest older producer wins: EX/MEM is checked before MEM/WB).

When an instruction that reads a register *r* is directly after a lw instruction that loads that register *r*, a stall is issued for one cycle to enable forwarding.

The data-memory interface is backpressured (valid/ready, with mem\_stall freezing every pipeline register) Three types are supported - fixed 1-cycle, fixed 3-cycle, and randomised 3-cycle (3-6 cycles). Branch resolution happens in EX. The branch is always predicted to NOT be taken so in the event of a taken JZ, two instructions are already loaded into ID and IF and they are squashed.

## Part B – SVA Specification and Rationale (in summary)

A final suite of 30 SystemVerilog assertions was developed and verified. To prove these assertions, they were subjected to three different verification methods:

1. **Simulation:** Tested in Verilator across all three memory latency configurations (1-cycle, fixed 3-cycle, and randomized).
2. **Formal Verification:** Analyzed using Bounded Model Checking (BMC) up to 14 cycles, allowing the formal solver to inject any valid instruction sequence and any memory stall duration.

3. **Mutation Testing:** Evaluated against a catalogue of intentionally injected hardware bugs to prove the suite’s fault-detection capabilities.

The assertions are organized into the same eight working groups across both the SV baseline and the Anvil-generated RTL. The SVA rationale pdf has detailed information about all the SVAs chosen.

## Part C — The Anvil Ports and Equivalence Testing

The Anvil implementation was developed across three distinct prototypes.

### The Three Anvil Ports

- **1. The Serialised Prototype:** The initial port relied on fully dynamic, two-phase channels throughout. Due to the two-way dynamic channels between all the phases, ID stage waits for an explicit acknowledgement from EX, which in turn is waiting on MEM and WB; this creates a global blocking chain where no stage can accept new data until the current instruction completely retires. Therefore technically this was not a “pipelined” processor as no two stages ever overlapped. It generated 2,625 lines of RTL and 52 distinct handshake wires.
- **2. The Static Baseline:** The second port replaces every inter-stage channel with a fully static @#1 – @#1 contract which guarantees that every instruction is only in a particular stage for exactly one cycle. This can handle forwarding, flushing and stalls. A stalled cycle re-issues the same PC and injects a bubble with `valid=0` (a NOP basically), so that the one cycle contract is not broken. This port compiles with every lifetime and borrow check on, and reaches 1 instruction per cycle similar to the SystemVerilog baseline. The limitation is that since the contract is static there is no room for variable backpressure in this processor
- **3. The Dynamic Backpressured Port (Current):** The third port implements the variable memory backpressure. Similar to the previous port it can handle forwarding, flushing and stalls. It is also only 1 IPC whenever memory is idle. The cost is that it compiles only with `-disable-lt-checks` (see Observation 1)

What lets this port pipeline at all, despite using the same dynamic primitive as Port 1, is what an acknowledgment is allowed to mean. In Port 1, a stage’s ack to its upstream stage is gated behind the entire downstream chain finishing:

```
send ep_out.req(data) >> // tell Memory
recv ep_out.res >> // wait for Memory to finish
send ep_in.res(1'b1) // only now tell Decode "I'm free"
```

so Decode cannot accept a new instruction until the current one has completely finished Writeback. Because this delay cascades backward through every stage, Fetch is physically blocked from pulling the next instruction until the current one has fully

finished. In Port 3, there is no explicit acknowledgement message; the handshake happens implicitly the moment the data is received:"

```
let data = recv ep_in.req >> // implicit ACK ("I'm free") happens here
set my_buf := data >> // captured it
send ep_out.req(*my_buf) // forward it whenever that resolves
```

Five stages can therefore each hold a different in-flight instruction at once, which is when Port 1 is at 11 CPI Port 3 is at 1 IPC.

Ultimately, Anvil forces you to choose: you can use static channels to get a perfectly safe pipeline that can't handle real memory, or you can use dynamic channels to handle real memory, but lose all of your formal safety guarantees.

### Equivalence Methodology

Instead of just checking the final output of a single basic program, the suite drives both processors simultaneously with identical inputs.

Eight targeted assembly programs were created (equivalence/assemble.py) to stress-test specific pipeline hazards. These tests cover: the original demonstration program, a branch target, a far unconditional jump, a four deep forwarding chain, a write to \$zero, a load into \$zero, a store-then-load, and a double load-use.

Each of the eight programs is loaded into both processors (the SystemVerilog baseline and the Anvil port) and simulated side-by-side. To verify perfect equivalence, three things were compared: the cycle-by-cycle Program Counter trace, the full 32-entry register file after the pipeline completely emptied, and the final state of the data memory. Across all eight test programs, the two processors matched perfectly on all three checks.

|                                      | SV baseline | Serialised Prototype<br>(dynamic, no b/p) | Static Baseline<br>(static @#1-@#1) | Dynamic Backpressured<br>(dynamic, current) |
|--------------------------------------|-------------|-------------------------------------------|-------------------------------------|---------------------------------------------|
| Cycles/instruction (memory idle)     | 1           | 11                                        | <b>1</b>                            | <b>1</b>                                    |
| Variable memory latency              | yes         | yes                                       | no                                  | yes                                         |
| Compiles with all lifetime checks on | n/a         | –                                         | <b>yes</b>                          | no                                          |
| Generated RTL (lines)                | –           | 2,625                                     | 974                                 | 1,474                                       |
| Distinct handshake wires             | –           | 52                                        | <b>0</b>                            | 59                                          |

Table 1: Headline metrics across all three Anvil ports compared to the SV baseline.

Instead of relying on internal wire probes, the test suite proves perfect equivalence by checking the observable state of both processors:

- **Cycle-Accurate Timing:** Comparing the Program Counter every single cycle proves that all stalls, squashes, and branches happen on the exact same clock edge in both designs.
- **Final Registers and Memory:** The full 32-entry register file and data memory after the pipeline completely emptied were compared. This guarantees the mathematical results are identical.

- **Implicit Forwarding Tests:** Anvil scrambles its internal wire names, making it impossible to probe the forwarding multiplexers directly. However, if the forwarding logic failed, our programs would have completely corrupted the final registers.

### The 1-Cycle Boot Offset

The cycle-by-cycle trace did surface one structural difference: Anvil takes one extra clock cycle after reset to load its instruction memory. This isn't a bug, it's a consequence of how each language treats reset. SV's `if (reset)` block is just ordinary procedural code inside a clocked process, so the same edge that zeroes the pipeline registers can also preload `imem` with real instruction content all in one atomic assignment set. Anvil's reset is a language primitive that only sets registers to their default (zero) value; there's no way to initialise it with non-default content at that same instant, so loading the program into `imem` has to happen as an explicit set in Fetch's first normal loop iteration, costing one real cycle before fetching can begin. Once the test script subtracted this 1-cycle offset, every single Program Counter trace across all eight test programs matched perfectly.

## Part D — Investigating Anvil: Three Observations

### Observation 1: What Anvil's timing-safety checks reject

The rejection reduces to ten lines (`lifetime_limit.anv`):

```
proc DynStage(ei: right dyn_in, eo: left dyn_out) {
  reg r : logic[32];
  loop {
    let x = recv ei.req >> // arrives at cycle T
    send eo.req(32'd0) >> // unbounded wait: completes T+k, k unknown
    set r := x // REJECTED
  }
}
```

In the above code the compiler says "Borrow checking failed" this is because it can't guarantee the correctness of `x` - that is it can't guarantee that `x` remains unchanged from the `let` statement to the `set` as there is an unbounded wait in between. Any pipeline stage that needs to receive data from multiple channels in the same clock cycle is going to crash the safety checker.

Anvil does have a feature to keep a value alive across an unknown wait by using a dependent contract, which tells the channel to keep the data alive until a "done" signal is sent. However, this feature only works if the wait and the release happen on the exact same channel. The compiler's math checks one channel at a time. In a pipeline, you receive data on one channel (the input) but wait on a completely different channel (the memory). Because there is no contract linking these two independent channels together, the safety checker fails.

## **Observation 2: What Anvil accepts that the SVA-derived tests catch**

When an instruction finishes, the Writeback stage sends its final data backward to two independent consumers: the Decode stage (to write to the register file) and the Execute stage (to forward to the ALU). Anvil uses two separate @#1 buses for this.

To test the compiler, one of the payloads was changed so that the two buses transmitted completely different data.

## **Observation 3: Silent Deadlocks and the Limits of the Type System**

In my first port - in the serialized prototype two way dynamic channels were used between all the stages. Fetch has to wait for Decode which has to wait for Execute but because Execute sends branch address to fetch it has to wait for ack from Fetch which results in a circular deadlock. The Anvil compiler completely missed this structural flaw. It compiled the code successfully with zero warnings, but the resulting simulation permanently froze at clock cycle zero.

## **Part E – What I would add to Anvil**

1. **Static deadlock detection:** The compiler could detect circular deadlocks (Observation 3) at compile time.
2. **An invariant over a data structure:** e.g. `reg regs with regs[0]==0`. The compiler could scan the logic, and if it finds any data path that could potentially alter `regs[0]`, it could throw a compile error and refuse to generate the RTL.

## **Provenance Note**

An AI assistant (Claude) was used to assist with this project. It helped draft and debug the Anvil code, review SVA logic, run the automated test scripts, and structure this report.

Every metric, line count, and test result presented in this document was manually verified directly against the final codebase and simulation logs to ensure complete accuracy. All architectural design decisions, interpretation of results are done by me.